

HAJIMI 智能桌面指引助手 —— 核心 AI 算法详解

文档版本：V1.0

定位：聚焦 HAJIMI 系统中真正涉及人工智能模型的算法，按处理阶段结构化组织，逐一给出"数学原理 / 架构 / 输入输出 / 训练与优化 / 在 HAJIMI 中的角色"五要素。

范围说明：纯工程类算法（如 SHA256 哈希、SQL 查询、状态机迁移）不在本篇重点，仅在涉及 AI 模型时点到为止。

〇、HAJIMI 中 AI 算法的全景图

按数据流向自底向上梳理，可将 HAJIMI 的 AI 算法划分为 5 个阶段、9 个核心模型：

阶段1 感知层 (Perception)
└ 1.1 YOLOv8 — 屏幕 UI 元素检测
└ 1.2 Florence-2 — 元素语义描述 (图标 captioning)
备选: GroundingDINO + PaddleOCR
阶段2 标注层 (Annotation)
└ 2.1 Set-of-Mark 渲染算法 (非神经网络, 但为 LLM 接口)
阶段3 理解层 (Understanding)
└ 3.1 jieba — 中文分词与词性标注 (统计模型)
└ 3.2 BERT-base-chinese — 九大意图域分类 (Transformer 编码器)
└ 3.3 综合置信度融合 (加权 + 阈值决策)
阶段4 规划层 (Planning)
└ 4.1 复杂度评分器 (规则评分 → 预留 BERT 分类器)
└ 4.2 GPT-4V / Qwen-VL-Max — 多模态任务规划 (Decoder-only)
└ 4.3 屏幕指纹比对 (SHA256 + Jaccard 相似度)
阶段5 语音层 (Speech)
└ 5.1 ASR: Vosk(离线) / 百度短语音 API(云端)
└ 5.2 TTS: pyttsx3(离线) / Azure TTS(云端)
阶段6 服务端 (Server-Side AI)
└ 6.1 模板匹配引擎 — jieba + TF-IDF + 余弦相似度

下面对每一模块进行**五要素式**详细展开。

阶段 1 感知层：让机器"看见"屏幕 UI

1.1 YOLOv8 —— 屏幕 UI 元素检测器

1.1.1 数学原理

YOLOv8 (You Only Look Once v8) 属于 **单阶段 (one-stage) 目标检测器**，其核心思想是把检测问题转化为一次回归。

输入： 图像 $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$

输出： 检测框集合 $\{(b_i, c_i, s_i)\}_{i=1}^N$ ，其中

- $b_i = (x_c, y_c, w, h)$ 为框几何参数,
- $c_i \in \{1, \dots, K\}$ 为类别索引,
- $s_i \in [0, 1]$ 为置信度。

核心数学表达:

1. **网格划分** (YOLOv8 沿用 YOLOv5 的 anchor-free 设计): 将输入图像划分为 $S \times S$ 网格 (如 20×20 步长), 每个网格预测若干候选框。
2. **回归目标** (对于每个候选框):

$$t_x = 2\sigma^{-1}(p_x) - 0.5 + c_x$$

$$t_y = 2\sigma^{-1}(p_y) - 0.5 + c_y$$

$$t_w = (2\sigma(p_w))^2 \cdot a_w$$

$$t_h = (2\sigma(p_h))^2 \cdot a_h$$

其中 (c_x, c_y) 是网格左上角坐标, a_w, a_h 是预设尺度, σ 为 Sigmoid。

3. **损失函数** (YOLOv8 采用 **TAL (Task-Aligned Assigner) + Distribution Focal Loss**):

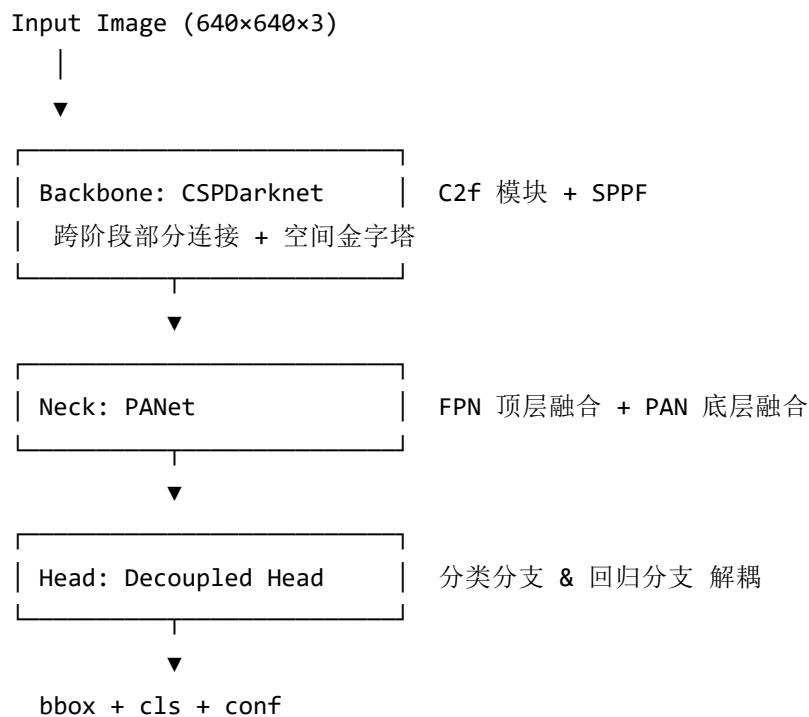
$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{box}} + \lambda_2 \mathcal{L}_{\text{cls}} + \lambda_3 \mathcal{L}_{\text{dfl}}$$

- $\mathcal{L}_{\text{box}}$: CloU Loss, 用于框回归;
- $\mathcal{L}_{\text{cls}}$: 二值交叉熵 (BCE);
- $\mathcal{L}_{\text{dfl}}$: Distribution Focal Loss, 把 bbox 边界建模为离散概率分布, 提升定位精度。

4. **NMS 后处理**: 用非极大值抑制去除冗余框。

1.1.2 网络架构

YOLOv8 整体由三部分组成:



关键创新点：

- **C2f 模块**：比 YOLOv5 的 C3 模块更轻量、特征更丰富；
- **Anchor-Free**：摆脱预设 anchor，泛化性更好；
- **Task-Aligned Assigner**：分类与回归任务正负样本对齐分配。

1.1.3 在 HAJIMI 中的训练/微调

OmniParser 团队使用的是在**67k 张屏幕截图**上微调的 YOLOv8m，类别约 10 类：

类别 ID	类别名	含义
0	button	按钮
1	input	输入框
2	icon	图标
3	menu	菜单项
4	checkbox	复选框
5	dropdown	下拉框
6	text	纯文字块
7	radio	单选
8	tab	标签页

类别 ID	类别名	含义
9	slider	滑块

训练数据来源：

- 公开数据集：RICO、Samsung UI、PubLayNet；
- 合成数据：HTML+CSS 渲染 + 自动标注（解决真实标注稀缺）；
- 人工微调：~5k 张。

训练超参（典型）：

超参	值
优化器	AdamW
初始学习率	1×10^{-3}
权重衰减	5×10^{-4}
Batch Size	64（单卡 A100）
Epochs	100
数据增强	Mosaic + MixUp + HSV
图像尺寸	640×640

1.1.4 在 HAJIMI 中的角色

维度	说明
位置	客户端感知层 PARSER 子模块
输入	mss 截取的屏幕图像（高 DPI 缩放后）
输出	元素列表 $E = \{(bbox_i, type_i, conf_i)\}$ ，与下一阶段 Florence-2 级联
性能	单张 1080p 截图 ~200ms（RTX 3060）
降级方案	无独显时切换为 GroundingDINO（精度略低，纯 CPU 可跑）

1.2 Florence-2 —— 元素语义描述模型

1.2.1 数学原理

Florence-2 是 Microsoft 在 2023 年提出的**统一视觉基础模型**，采用 **Encoder-Decoder Transformer** 架构，专门为"多任务视觉理解"设计。

核心思想：用统一的序列到序列（Seq2Seq）框架，把所有视觉任务（检测、描述、分割、检索）都表达为"图像 → token 序列"任务。

架构：

$$\begin{aligned} \text{Visual Tokens} &= \text{DaViT Encoder}(\mathbf{I}) \\ \text{Output Tokens} &= \text{Transformer Decoder}(\text{Visual Tokens}, \text{Task Prompt}) \end{aligned}$$

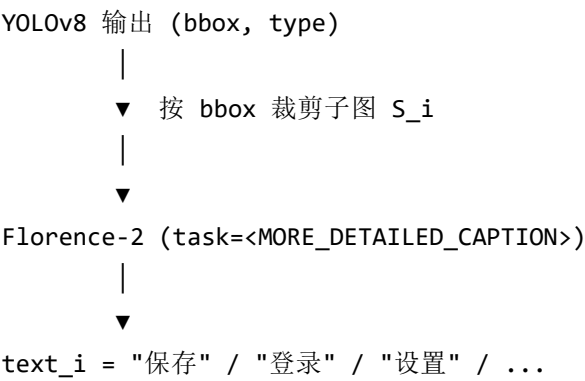
其中 **DaViT（Dual Attention ViT）** 同时使用空间注意力和通道注意力。

关键技术：

- 1. **Task Prompt Token**：不同任务用不同 token 引导，如 <CAPTION>、<DETECTION>、<OD>（Object Detection）。
- 2. **统一数据格式**：所有任务输出格式为 <loc_x1><loc_y1><loc_x2><loc_y2><text> 。
- 3. **预训练数据集 FLD-5B**：5B 注释的图像-文本对（远大于 CLIP 的 400M）。

1.2.2 在 HAJIMI 中的角色

Florence-2 在 HAJIMI 中**不是独立调用**，而是作为 OmniParser V2 流水线的"语义描述"环节：



输入：单个元素子图 S_i （裁剪自 YOLOv8 bbox）
输出：文字标签 t_i ，如 "Save"、"Username input"、"齿轮 设置图标"

为什么用 Florence-2 而不是 BLIP-2：

- 对小图标（≤32×32）的识别 F1 显著高于 BLIP-2；
- 推理速度接近（~80ms / 图）；

- 统一任务接口便于以后扩展。

1.2.3 训练/微调策略

OmniParser 团队在 Florence-2-base 上针对 UI 元素 caption 任务进行微调：

超参	值
任务提示符	<MORE_DETAILED_CAPTION>
训练集	67k 截图 + 对应元素 caption
学习率	3×10^{-5}
Batch	32
Epochs	20
Loss	交叉熵 (decoder 端)

1.3 GroundingDINO + PaddleOCR（备选方案）

当本地无独显、OmniParser 无法运行时，HAJIMI 切换为**双模型组合**：

1.3.1 GroundingDINO（开放词汇目标检测）

- **架构**：基于 DINO 检测器 + 文本引导（CLIP 文本编码器），输入文本 prompt"button"即可检测所有按钮。
- **数学核心**：跨模态特征融合

$$F_{\text{fuse}} = \text{Deformable Cross-Attention}(F_{\text{img}}, F_{\text{text}})$$

- **优势**：不需要预定义类别，可自定义 prompt（如 "settings icon")；
- **劣势**：速度比 YOLOv8 慢（~400ms/张），适合低频调用。

1.3.2 PaddleOCR（文字检测+识别）

- **检测**：DB（Differentiable Binarization）网络
- **识别**：CRNN（CNN+RNN+CTC）序列模型
- **在 HAJIMI 中的作用**：识别按钮/输入框上的**文字标签**，作为元素的 text 字段补充。

阶段 2 标注层：Set-of-Mark (SoM) 渲染

2.1 SoM 的数学表达

SoM 严格来说不是一个神经网络，而是 **LLM 接口协议**。但其在算法上等价于一次"区域划分 + 可视化编码"过程。

形式化定义：

给定图像 $\mathbf{I}$ 与元素集合 $E = \{(b_i, c_i, t_i)\}_{i=1}^N$ ，SoM 算法 S 生成标注图 $\mathbf{I}_{\text{som}}$ 与映射表 M ：

$$\mathbf{I}_{\text{som}}, M = S(\mathbf{I}, E)$$

其中 $M : \text{id}_i \mapsto (b_i, c_i, t_i, \text{center}_i)$ 。

关键设计：

- 1. **颜色编码**：不同类型用不同色（按钮蓝、输入框绿、图标黄.....）
- 2. **数字编号**： $\text{id}_i = \sim + \text{序号}$ ，强制独占编号
- 3. **区域完整性**：裁剪框 = YOLOv8 bbox + 4px 外扩

为什么有效（信息论视角）：

LLM 的"输出坐标"任务可视为一次 **量化编码**——把连续空间 $\mathbb{R}^{H \times W}$ 编码为 64KB 的 token。SoM 把任务转化为 **离散选择**（"从 N 个候选中选 1 个"），大大降低了输出熵：

$$H(\text{output}) = \log_2 N \ll H(\text{坐标})$$

这就是 SoM 把准确率从 ~40% 提升到 ~95% 的本质原因。

2.2 在 HAJIMI 中的工程化

完整伪代码: HAJIMI SOM 模块

```
def generate_som(image, elements):
    canvas = image.copy()
    drawer = ImageDraw.Draw(canvas)
    mapping = {}

    for i, (bbox, etype, text) in enumerate(elements, 1):
        eid = f"~{i}"
        color = TYPE_COLOR[etype]

        # 1. 画彩色边界框
        drawer.rectangle(bbox, outline=color, width=3)

        # 2. 顶部贴编号标签
        lx, ly = bbox.cx - 22, bbox.y1 - 34
        drawer.rounded_rectangle(
            (lx, ly, lx+44, ly+30),
            radius=4, fill='white', outline=color, width=2
        )
        drawer.text((lx+8, ly+4), eid, fill=color, font=bold_16)

        # 3. 记录映射
        mapping[eid] = {
            'bbox': bbox, 'type': etype, 'text': text,
            'center': bbox.center
        }

    return encode_base64(canvas), mapping
```

输出:

- 标注图 → Base64 编码 → 送入多模态 LLM;
- 映射表 → 内存中的 dict → 用于 ANNO 坐标反查。

阶段 3 理解层：让机器"听懂"用户

3.1 jieba 中文分词与词性标注

3.1.1 数学原理

jieba 的核心是基于 **HMM（隐马尔可夫模型）+ DAG（无环图）+ 动态规划** 的混合分词算法。

算法步骤：

- 1. **构造 DAG**：扫描句子，对于位置 i 到 j 的子串 $[i, j]$ ，查找词典中是否成词，构造有向无环图。
- 2. **动态规划求最大概率路径**：

定义从句子右端到左端的最大概率路径：

$$DP[i] = \max_{j:[i,j] \in \text{词典}} (P([i, j]) \times DP[j])$$

其中 $P(w) = \frac{\text{freq}(w)}{\text{total_words}}$ 为词频概率。

- 3. **未登录词识别**：对词典中未出现的字符序列，用 HMM 模型的 Viterbi 算法识别为人名、地名等。

HMM 三要素：

- 状态集合 $S = \{B, M, E, S\}$ （词的开始、中间、结束、单字成词）
- 发射概率 $P(\text{char}|\text{state})$
- 转移概率 $P(\text{state}_t|\text{state}_{t-1})$

3.1.2 在 HAJIMI 中的角色

用途	作用
词性标注	提取动词 (v) + 名词 (n) → 核心目标
停用词过滤	去除 "的""了""啊" 等
约束提取	否定 ("不""别")、位置 ("在""到")
模板关键词	提取 trigger keywords（服务端模板匹配）

3.1.3 训练/优化

jieba 是预训练统计模型，HAJIMI 主要做**词典微调**：

```
# 加载业务词典
jieba.load_userdict("hajimi_dict.txt")
# 词典条目: "保存按钮 10 n", "Word文档 10 n", ...
```

词典按词频 + 词性 + 权重加载，分词概率更新：

$$P'(w) = \frac{\text{freq}(w) + \alpha \cdot \mathbf{1}_{w \in \text{userdict}}}{\text{total} + \alpha \cdot |\text{userdict}|}$$

3.2 BERT-base-chinese —— 九大意图域分类器

3.2.1 数学原理

BERT (Bidirectional Encoder Representations from Transformers) 是一个深层 **Transformer** 编码器。

输入：分词后的 token 序列 $T = [t_1, t_2, \dots, t_n]$

输出：上下文相关的向量表示

$$\mathbf{H} = \text{BERT}(T) \in \mathbb{R}^{n \times d}$$

其中 $d = 768$ (base 版本)。

Transformer 核心 (自注意力机制)：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

多头版本：

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

分类头 (HAJIMI 在 BERT 之上接一个 9 类 softmax 分类器)：

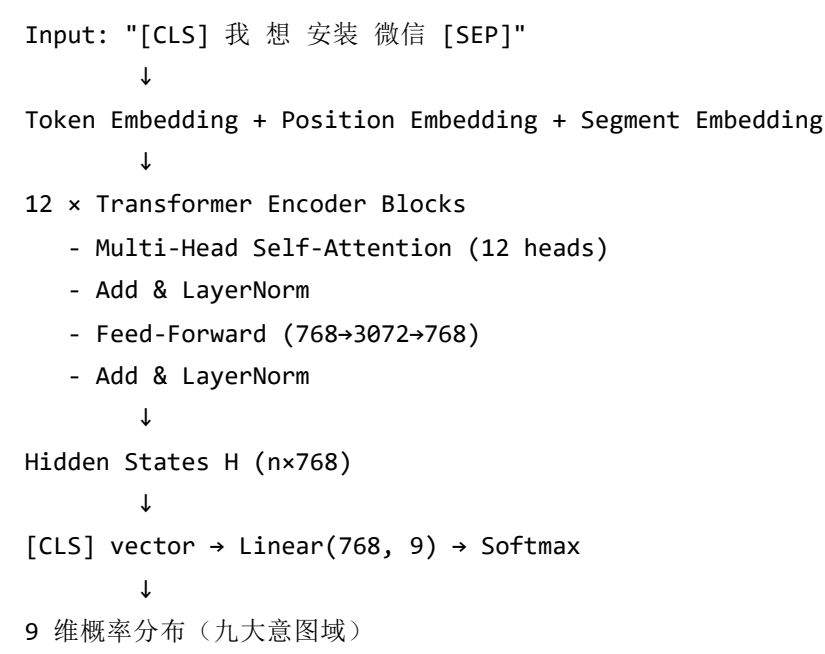
$$P(c|T) = \text{softmax}(\mathbf{W}_c \cdot \mathbf{H}_{[CLS]} + b_c)$$

其中 $\mathbf{H}_{[CLS]}$ 是 [CLS] 标记的隐状态 (被视为整句表示)。

损失函数：标准交叉熵

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^9 y_{i,c} \log P(c|T_i)$$

3.2.2 网络架构



关键设计：

- **双向自注意力**：可同时看到左右上下文；
- **12 层堆叠**：第 1 层捕捉词级信息，逐层抽象到句级；
- **预训练任务**：MLM（掩码语言模型）+ NSP（下一句预测）；
- **微调成本小**：~3 epoch 即可在 5k 标注数据上达到 85% 准确率。

3.2.3 训练/微调

阶段	细节
预训练	HuggingFace bert-base-chinese （Google 官方中文版本，已含 MLM+NSP 预训练）
微调数据	5,000 条人工标注 query，9 类均衡分布
微调超参	优化器 AdamW，LR=2e-5，BERT 衰减 0.95，Batch=32，Epochs=3，MaxLen=64
数据增强	同义词替换（"安装"→"装上"）、回译、句式改写
评估指标	9 类 macro-F1 ≥ 0.85，混淆矩阵人工复核
部署	ONNX 量化 → 端侧推理 ~15ms（i5 CPU）

九大概图域枚举（来自设计文档）：

- 0 操作执行指引 ("怎么保存", "如何打印")
- 1 界面元素认知 ("这个图标是什么", "这里写什么")
- 2 异常诊断与恢复 ("怎么打不开了", "出错了")
- 3 界面导航与定位 ("文件在哪", "设置在哪里")
- 4 内容认知与信息处理 ("这段话什么意思", "翻译一下")
- 5 桌面环境与资产管理 ("C盘空间", "内存占用")
- 6 状态监控与主动预警 ("CPU 高吗", "网络断了")
- 7 流程复盘与教学记忆 ("上一步怎么做的")
- 8 情感陪伴与语境缓解 ("我不会啊", "卡住了")

3.2.4 在 HAJIMI 中的角色

维度	说明
位置	客户端理解层 INTENT 子模块第一层
输入	jieba 分词后的 token 序列
输出	9 维概率分布 $\rightarrow \arg \max +$ 最大概率 p_{intent}
性能	推理 ~15ms (CPU) / ~5ms (GPU)
降级	模型未加载时回退到规则关键词匹配

3.3 综合置信度融合与主动澄清

3.3.1 数学定义

三层消歧输出的三个概率融合为一个综合置信度：

$$\text{Conf}_{\text{combined}} = 0.4 \cdot p_{\text{intent}} + 0.4 \cdot p_{\text{referent}} + 0.2 \cdot p_{\text{context}}$$

各项来源：

符号	含义	来源
p_{intent}	意图分类最大概率	BERT softmax 输出
p_{referent}	指代消解最大置信度	5 种消解器各自的归一化分
p_{context}	上下文一致性	上一轮指代与本轮的语义相似度（Sentence-BERT 编码 + 余弦）

3.3.2 决策规则

$$\text{Action} = \begin{cases} \text{直接执行} & \text{if } \text{Conf}_{\text{combined}} \geq 0.8 \\ \text{主动澄清} & \text{if } \text{Conf}_{\text{combined}} < 0.8 \end{cases}$$

3.3.3 主动澄清的问题生成算法

```
def generate_clarification(intent, candidates):
    if len(candidates) >= 2 and intent == '操作指引':
        # 二选一
        opts = [f"「{som_map[c].text}」" for c in candidates[:3]]
        return f"您是想 {'还是'.join(opts)} 吗? "
    elif intent == '内容认知':
        return "请补充一下您想了解什么内容? "
    else:
        return "我没有完全理解，请再说一次。"
```

澄清后：用户回答作为新事实锚点，追加到 MEMORY，重置置信度评估。

阶段 4 规划层：让机器"规划"任务

4.1 复杂度评分器（L2/L3 路由判别）

4.1.1 规则评分公式

$$\text{score}(Q) = \alpha \cdot \text{len}(Q) + \beta \cdot \#\text{verb}(Q) + \gamma \cdot 1_{\text{crossApp}}(Q)$$

典型系数（默认值，可热部署覆盖）：

系数	值	含义
α	0.3	句子长度因子（每字符加分）
β	8	动词数量因子（每动词加分）
γ	15	跨应用开关（"在...里""切换到..." 触发）

判定规则：

$$\text{PlanType} = \begin{cases} \text{L2 (快路径)} & \text{if } \text{score} < 30 \\ \text{L3 (慢路径)} & \text{if } \text{score} \geq 30 \end{cases}$$

4.1.2 未来升级路径（ComplexityRouter 抽象接口）

```
class ComplexityRouter(ABC):
    @abstractmethod
    def score(self, query: str) -> float:
        pass

class RuleRouter(ComplexityRouter):
    """当前实现"""
    def score(self, query): ...

class BertRouter(ComplexityRouter):
    """未来实现：微调 BERT 分类器"""
    def score(self, query):
        vec = bert_tokenizer(query)
        prob = self.bert_classifier(vec)
        # 二分类: simple / complex
        return prob[1] * 100 # 0~100
```

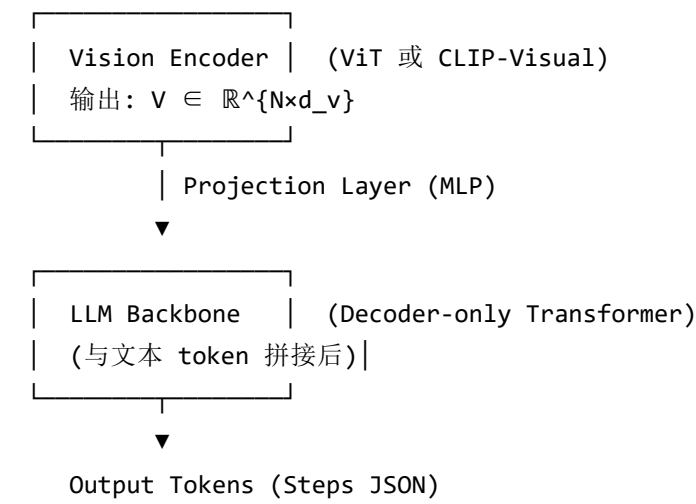
通过依赖注入切换实现，**业务代码零改动**。

4.2 多模态大模型：GPT-4V / Qwen-VL-Max（任务规划器）

4.2.1 数学原理

GPT-4V / Qwen-VL-Max 属于 **Decoder-only Transformer + 视觉编码器** 的多模态大模型。

整体架构：



关键数学组件：

1. **视觉编码器**：ViT 把图像切成 14×14 patches，展平后线性投影 + Position Embedding，送入多层 Transformer。
2. **跨模态投影**：

$$\mathbf{H}_{\text{mm}} = \text{Concat}(\mathbf{V} \cdot \mathbf{W}_v, \mathbf{T})$$

其中 $\mathbf{W}_v$ 是 MLP 投影矩阵， $\mathbf{T}$ 是文本 token embedding。

3. **自回归解码**：

$$P(t_i | t_{<i}, \mathbf{H}_{\text{mm}}) = \text{softmax}(\text{LM Head}(\mathbf{H}_i))$$

4. **训练目标**：标准语言模型交叉熵

$$\mathcal{L} = - \sum_{i=1}^L \log P(t_i^* | t_{<i}, \mathbf{H}_{\text{mm}})$$

其中 t_i^* 是 ground truth token 序列（如 JSON 化的步骤列表）。

4.2.2 在 HAJIMI 中的输入输出

输入（多模态 Prompt 构造）：

```
{
  "model": "gpt-4v",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "image",
          "image": "data:image/png;base64,{som_image}"
        },
        {
          "type": "text",
          "text": "你是桌面指引助手。请根据用户问题与屏幕标注图（Set-of-Mark），\n输出从当前状态到达目标的"
        }
      ]
    }
  ],
  "max_tokens": 2000,
  "temperature": 0.2
}
```


输出（LLM 生成的 JSON）：

```
{
  "steps": [
    {"idx": 1, "action": "点击左上角「文件」菜单", "target_element_id": "~5"},
    {"idx": 2, "action": "选择「保存」选项", "target_element_id": "~12"},
    {"idx": 3, "action": "在弹窗中点击「确定」", "target_element_id": "~20"}
  ]
}
```

4.2.3 训练/优化机制

HAJIMI 不参与 GPT-4V / Qwen-VL-Max 的训练（无算力），只做 Prompt Engineering + 上下文示例 (Few-shot)：

优化手段	说明
System Prompt	固定角色定义 + 输出格式约束
Few-shot 示例	注入 2~3 个 "query→steps" 范例
Temperature=0.2	低温度保证步骤稳定性
JSON Schema 约束	OpenAI JSON mode / Qwen structured output
Token 预算	max_tokens=2000，prompt<8k

4.2.4 模型架构对比

模型	提供方	参数量	多模态能力	HAJIMI 中的角色
GPT-4V / GPT-4o	OpenAI	未公开	顶级	L3 主用，云端调用
Qwen-VL-Max	阿里云	~72B	优秀	备选，国内可用
Claude 3.5 Sonnet	Anthropic	未公开	顶级	Adapter 抽象层支持
Qwen2-VL-2B	阿里	2B	中等	L2 快路径专用，本地 CPU 可跑

Adapter 模式实现多模型无缝切换：

```

class LLMAdapter(ABC):
    @abstractmethod
    def multimodal_plan(self, image, prompt) -> str: pass

class GPT4VAdapter(LLMAdapter):
    def multimodal_plan(self, image, prompt):
        return openai_client.chat.completions.create(
            model="gpt-4o",
            messages=[{"role": "user", "content": [
                {"type": "image_url", "image_url": image},
                {"type": "text", "text": prompt}
            ]}]
        )

class QwenVLAdapter(LLMAdapter):
    def multimodal_plan(self, image, prompt):
        return dashscope.MultiModalConversation.call(
            model="qwen-vl-max",
            messages=[{"role": "user", "content": [
                {"image": image}, {"text": prompt}
            ]}]
        )

```

4.3 屏幕指纹匹配算法

4.3.1 数学定义

指纹构成 (4 字段 + 1 哈希):

$F = \text{SHA256}(\text{WindowHandle} \oplus \text{TitleCategory} \oplus \text{Top5ElementTypes} \oplus \text{ForegroundProcess})$

字段级 Jaccard 相似度 (用于指纹比对):

$$\text{sim}(F_a, F_b) = \frac{|F_a.\text{fields} \cap F_b.\text{fields}|}{|F_a.\text{fields} \cup F_b.\text{fields}|}$$

判定规则:

$$\text{Progress} = \begin{cases} \text{推进} & \text{if sim} \geq 0.8 \\ \text{挂起} & \text{if sim} < 0.8 \end{cases}$$

4.3.2 算法实现

```
import hashlib

def compute_fingerprint(som_map, window_handle, title, fg_process):
    # 1) 字段规整化
    title_cat = categorize_title(title)          # "Word 文档" / "Chrome 浏览器" ...
    top5_types = sorted([e['type'] for e in som_map.values()])[:5]
    top5_str = ','.join(top5_types)

    # 2) 拼接
    raw = f"{window_handle}|{title_cat}|{top5_str}|{fg_process}"

    # 3) SHA256
    return hashlib.sha256(raw.encode()).hexdigest()[:32] # 16字节即可

def match_fingerprint(actual, expected):
    # 字段级集合比较
    a_fields = set(parse_fields(actual))
    e_fields = set(parse_fields(expected))
    jaccard = len(a_fields & e_fields) / len(a_fields | e_fields)
    return jaccard >= 0.8
```

为什么是 Jaccard 而不是直接 SHA256 比对？

- 屏幕变化是**渐变**的（按钮文字变了、菜单项多了），哈希值会完全不同；
- 字段级比对更鲁棒，只要"窗口类型 + 关键元素集合"一致即视为有效。

4.3.3 在 HAJIMI 中的角色

维度	说明
位置	客户端规划层 StateMachine 子模块
输入	当前屏幕元素 + 窗口信息
输出	32 字节指纹 + 推进/挂起布尔
性能	< 50ms / 次

阶段 5 语音层：让机器"说"和"听"

5.1 ASR（自动语音识别）

5.1.1 Vosk（离线方案）

架构：基于 Kaldi 的端到端语音识别（基于 CTC + n-gram 语言模型）。

核心数学（CTC：Connectionist Temporal Classification）：

给定音频特征序列 $X = (x_1, \dots, x_T)$ ，输出文字序列 $Y = (y_1, \dots, y_U)$ ($U \ll T$)：

$$P(Y|X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi|X) = \sum_{\pi} \prod_{t=1}^T P(\pi_t|x_t)$$

其中 π 是对齐路径， $\mathcal{B}$ 是合并重复 + 去 blank 的操作。

HAJIMI 中的角色：

- 中文小模型 ~50MB，本地推理；
- 触发：麦克风按钮按下 → 录音 → 松开/静默 2s → 转文字；
- 输出：字符串 → 自动填入输入框。

5.1.2 百度短语音 API（云端方案）

基于 **Transformer Transducer** 或 **Conformer** 架构的工业级模型（具体架构不公开）。HAJIMI 通过 RESTful API 调用。

5.2 TTS（语音合成）

5.2.1 pyttsx3（离线方案）

本质：Windows SAPI5 / Linux eSpeak 的 Python 封装，**不涉及 AI 模型**。基于预录制的语音单元拼接合成，质量一般但零延迟、零依赖。

5.2.2 Azure TTS（云端方案）

基于 **Transformer TTS / VITS** 等神经语音合成模型。

核心数学（以 VITS 为代表）：

VITS = **VAE + Flow + GAN + GAN-TTS** 的多任务联合训练：

Encoder : $z = \text{TextEncoder}(text)$
 VAE : $P(z|text) \approx \mathcal{N}(\mu, \sigma)$
 Decoder : $\hat{x} = \text{Decoder}(z, F0)$ (音频波形)
 Discriminator : $D(\hat{x}, x)$ 对抗训练

HAJIMI 中的角色：

- 每步指引生成完毕后自动播报；
- 语速 0.5~1.5 倍可调；
- 与 ANNO/TEXT 同步触发。

阶段 6 服务端 AI 算法：模板匹配

6.1 TF-IDF + 余弦相似度

6.1.1 数学原理

TF-IDF (词频-逆文档频率)：

$$\text{tfidf}(w, d) = \text{tf}(w, d) \times \text{idf}(w) = \frac{n_{w,d}}{\sum_{w'} n_{w',d}} \times \log \frac{N}{1 + |\{d : w \in d\}|}$$

其中 N 是模板总数， $n_{w,d}$ 是词 w 在文档 d 中的频次。

余弦相似度：

$$\text{sim}(q, t) = \frac{\mathbf{q} \cdot \mathbf{t}}{\|\mathbf{q}\|_2 \cdot \|\mathbf{t}\|_2} = \frac{\sum_{i=1}^V q_i t_i}{\sqrt{\sum_i q_i^2} \cdot \sqrt{\sum_i t_i^2}}$$

6.1.2 匹配流程

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

class TemplateMatcher:
    def __init__(self, templates):
        self.templates = templates
        corpus = [' '.join(jieba.cut(t['trigger_keywords']))
                  for t in templates]
        self.vectorizer = TfidfVectorizer(max_features=5000)
        self.matrix = self.vectorizer.fit_transform(corpus)

    def match(self, query):
        tokens = ' '.join(jieba.cut(query))
        q_vec = self.vectorizer.transform([tokens])
        sims = cosine_similarity(q_vec, self.matrix)[0]
        best = sims.argmax()
        if sims[best] >= 0.90:
            return self.templates[best], sims[best]
        return None, 0.0
```

性能优化：

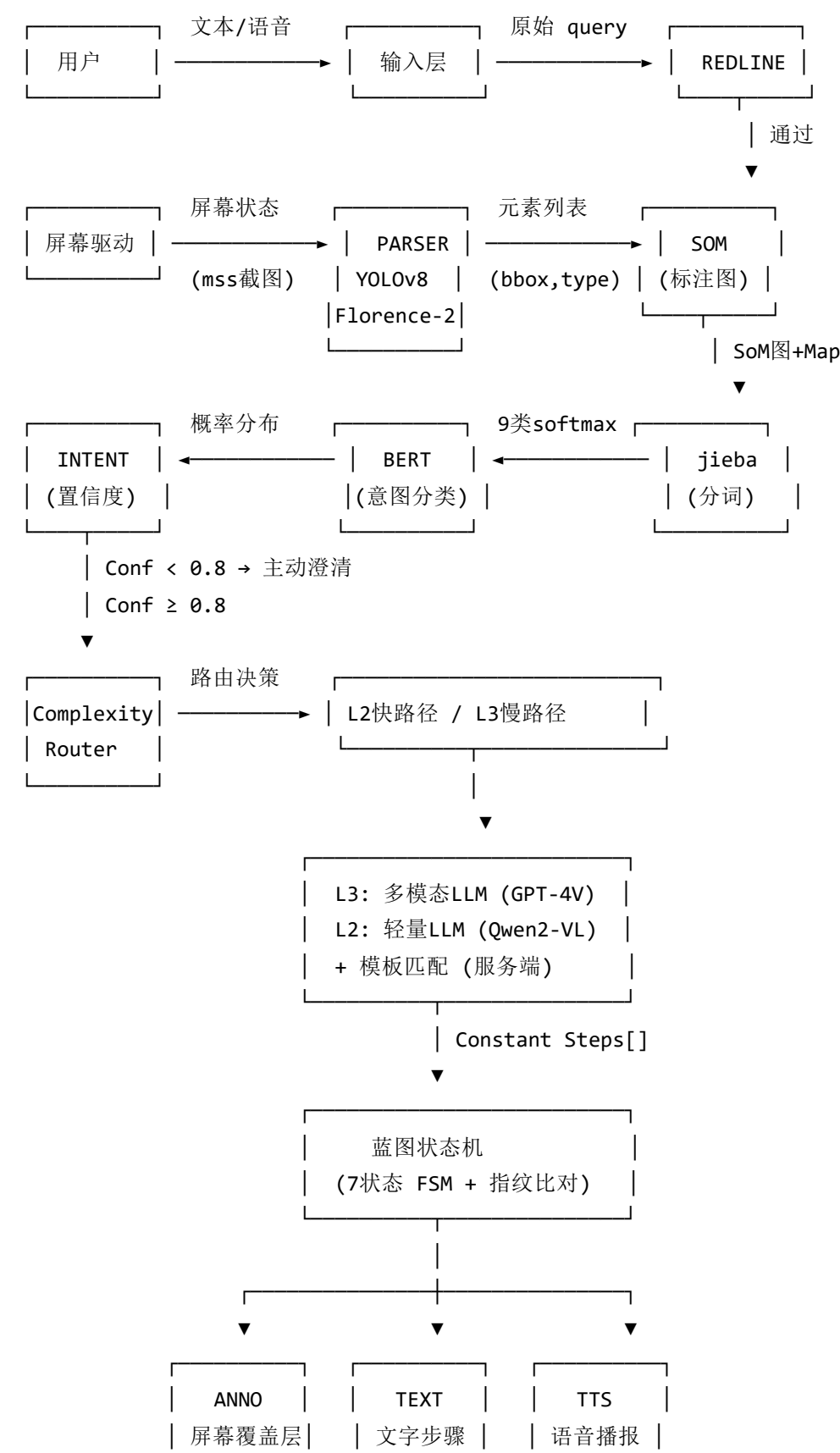
- PostgreSQL GIN 索引加速关键词倒排；
- TF-IDF 矩阵常驻内存（~5MB / 1万模板）；
- 单次匹配 < 100ms（P99 < 150ms）。

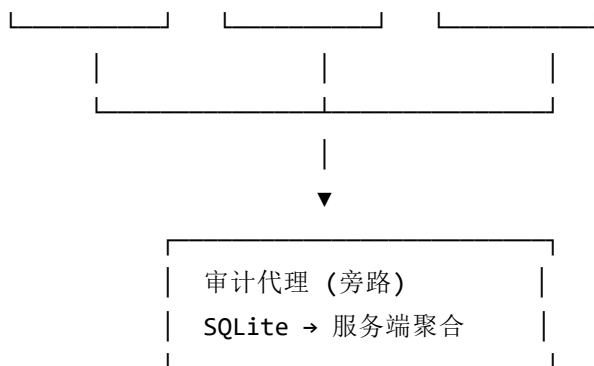
6.1.3 在 HAJIMI 中的角色

维度	说明
位置	服务端 API 接口层 /api/templates/match
输入	用户 query 字符串
输出	命中模板（蓝图 + 相似度）或 None
作用	L3 路径的 加速器 ：命中时直接返回蓝图，跳过 LLM 调用（节省 3~7s）

阶段 7 算法模块间交互与数据流

7.1 全链路数据流图





7.2 数据格式在各模块间的约定

模块间	数据格式	示例
输入 → REDLINE	str (raw query)	"怎么保存这个文档？"
REDLINE → INTENT	str / None	"怎么保存这个文档？"
INTENT → CAP	bool (need_screen)	True
CAP → PARSER	PIL.Image	Image(1920, 1080, RGB)
PARSER → SOM	List[Element]	[{bbox, type, text, conf}, ...]
SOM → LLM	(base64_img, Map)	"data:image/png;base64,..."
LLM → PLANNER	JSON str	{"steps":[{"idx, action, target_element_id}]}
PLANNER → ANNO	List[Step]	[{action, target_element_id="~3"}]
ANNO → User	QPixmap 渲染	屏幕上的红框+箭头+标签
PLANNER → AUDIT	JSON dict	{task_id, steps, fp_log, ...}
AUDIT → SERVER	HTTPS POST	脱敏后的批量事务

7.3 算法间的协同机制

7.3.1 共享上下文：MEMORY 模块

所有算法模块共享一个上下文对象，由 MEMORY 统一管理：


```
class Context:
    short_term: deque # 最近3轮对话
    blueprint: Optional[Blueprint] # 当前蓝图
    anchors: List[Anchor] # 事实锚点
    user_prefs: dict # 用户偏好
```

写入:

- 用户每次提问 → 推入 short_term;
- LLM 澄清回答 → 追加到 anchors;
- L3 蓝图生成 → 写入 blueprint。

读取:

- INTENT 参考 anchors 提升 $p_{context}$;
- PLANNER 参考 short_term 补充 Prompt;
- ANNO 参考 blueprint 获取当前步。

7.3.2 异步并行: 执行层

ANNO / TEXT / TTS **并行触发** (不是串行):

```
import asyncio

async def execute_step(step):
    await asyncio.gather(
        anno.render_async(step.target), # 屏幕覆盖
        text.update_async(step),        # 文字步骤
        tts.speak_async(step.action),    # 语音播报
    )
```

7.3.3 旁路异步: 审计

审计代理**完全独立于主链路**, 通过 ThreadPoolExecutor 异步执行:

```
class AuditAgent:
    def __init__(self):
        self.executor = ThreadPoolExecutor(max_workers=2)

    def report_async(self, transaction):
        self.executor.submit(self._report, transaction)

    def _report(self, transaction):
        # 1) 写入本地 SQLite
        # 2) 隐私脱敏
        # 3) 批量 POST（累积10条触发）
        # 4) 失败重试
        ...
```

即使审计完全失败，主链路也不受影响。

阶段 8 训练 / 优化 / 部署全流程

8.1 各模型的训练责任划分

模型	谁训练	HAJIMI 中做什么
YOLOv8（OmniParser）	Microsoft + 社区	直接调用预训练权重，按需 ONNX 量化
Florence-2	Microsoft	同上
BERT-base-chinese	Google（预训练）	HAJIMI 团队微调 （5k 标注，9 分类）
GPT-4V / Qwen-VL-Max	OpenAI / 阿里	不训练 ，只做 Prompt Engineering
Qwen2-VL-2B	阿里	直接调用
Vosk	Alpha Cephei	直接调用
Azure TTS	Microsoft	直接调用
TF-IDF	无（统计方法）	HAJIMI 团队在历史 query 上离线训练

8.2 模型部署方式

客户端进程内 (PyTorch / ONNX Runtime / API)		
└ BERT (微调后, ONNX, 8MB)	~15ms / 推理	
└ jieba (统计, 5MB)	~5ms	
└ pyttsex3 (本地 DLL)	启动 50ms	
客户端子进程 (OmniParser, GPU 隔离)		
└ YOLOv8m (ONNX, 50MB)	~200ms / 帧	
└ Florence-2 (ONNX, 270MB)	~80ms / 图	
云端 API (HTTPS)		
└ GPT-4V / Qwen-VL-Max	3~7s / 规划	
└ Vosk API (可选)	~1s / 识别	
└ Azure TTS API (可选)	~300ms / 句	
服务端 (FastAPI)		
└ TF-IDF 匹配 (sklearn, 内存常驻)	< 100ms	

8.3 模型的在线学习 / 持续优化

HAJIMI 具备持续优化能力（虽然不在第一版实现）：

优化点	数据来源	机制
BERT 分类器增量训练	用户反馈 + 人工标注	每月一次 retrain, ONNX 重新量化
LLM Prompt 优化	失败归因看板	管理员根据 TOP10 失败调整 prompt
模板库扩充	用户"有用"反馈	管理员审核后发布至全局库
OmniParser 升级	Microsoft 新版本	OTA 替换权重文件
复杂度评分升级	标注数据	Rule → BERT 二分类

〇、总结：HAJIMI AI 算法的"9 模型 + 1 协议"

9 个核心 AI 模型

#	模型	类型	阶段	任务
1	YOLOv8	单阶段检测 CNN	感知	UI 元素定位
2	Florence-2	Encoder-Decoder Transformer	感知	元素语义描述
3	Set-of-Mark	接口协议（不是模型）	标注	视觉提示渲染
4	jieba	统计 + HMM	理解	中文分词词性
5	BERT-base-chinese	Encoder Transformer	理解	9 类意图分类
6	GPT-4V / Qwen-VL-Max	Decoder Transformer + 视觉	规划	多模态任务分解
7	Qwen2-VL-2B	轻量多模态	规划	L2 快路径推理
8	Vosk	CTC + n-gram	语音	离线 ASR
9	TF-IDF + 余弦	经典 IR 算法	服务端	模板匹配

1 个核心协议

SoM 元素 ID 协议：作为"通用语"贯穿感知→标注→LLM→执行→审计全链路，将 LLM 难以胜任的"输出像素坐标"任务转化为擅长的"输出编号"任务，准确率从 ~40% 跃升至 ~95%。

算法协同的"3+1 闭环"

闭环 1：感知 ↔ 标注 (YOLOv8 → SoM)	像素 → 编号
闭环 2：理解 ↔ 规划 (BERT → LLM)	意图 → 步骤
闭环 3：执行 ↔ 验证 (ANNO → Fingerprint)	标注 → 推进
+ 服务端闭环：客户端 → 模板 → 全局复用	一次学会，全域共享

这一算法体系既体现了**对前沿 AI 技术的工程化整合能力**（多模态 LLM、视觉基础模型、Transformer），也保留了对**经典算法的尊重**（TF-IDF、Jaccard、SHA256、jieba HMM）——工程落地的核心是**"合适的方法用在合适的环节"**。

文档结束。如需进一步了解任一算法的实现细节，可结合设计文档对应章节（§4.2 感知层 / §4.3 理解与规划层 / §4.4 执行层 / §4.6 服务端）与源代码进行交叉验证。